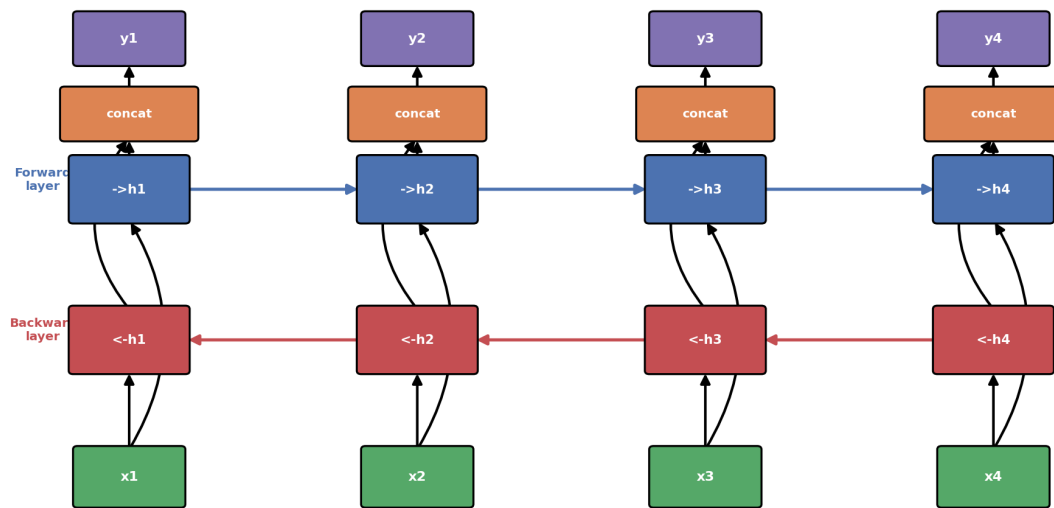


Bidirectional RNN

BiLSTM & BiGRU

Why We Need Them, How They're Architected, and Their Trade-offs

Bidirectional RNN - Full Architecture

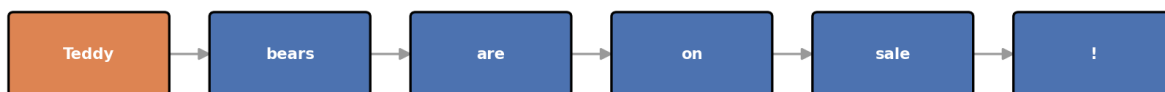


Covers: Why bidirectional? • Architecture • Advantages & drawbacks • BiLSTM • BiGRU

1. Why Bidirectional?

A standard (unidirectional) RNN, LSTM, or GRU only ever looks **backward in time** - at each step, the hidden state carries information from earlier in the sequence, but has no idea what comes next. For many tasks, this is a real limitation, because the correct interpretation of a word or element often depends on context that comes **after** it, not just before.

Why Bidirectional? A Forward-Only RNN Can't Resolve 'Teddy' at the Time It's Read



"Teddy" refers to a TOY - clear from words AFTER it



"Teddy" refers to a PERSON - clear from words AFTER it

Figure 1: The word 'Teddy' is ambiguous until you see what follows it - a forward-only RNN can't resolve this in time.

Classic examples where future context changes the meaning:

- **Named entity recognition:** "Teddy bears are on sale" (object) vs. "Teddy Roosevelt was president" (person).
- **Part-of-speech tagging:** a word's grammatical role can depend on what follows it.
- **Machine translation & summarization:** understanding the full sentence structure before deciding how to translate an early word.
- **Speech recognition:** disambiguating a phoneme using the sounds that come after it.

A **Bidirectional RNN (BiRNN)** solves this by processing the sequence in **both directions** - once forward (left to right) and once backward (right to left) - then combining the two views at every time step, so each position has access to the entire sequence's context.

2. Core Idea: Two Passes Instead of One

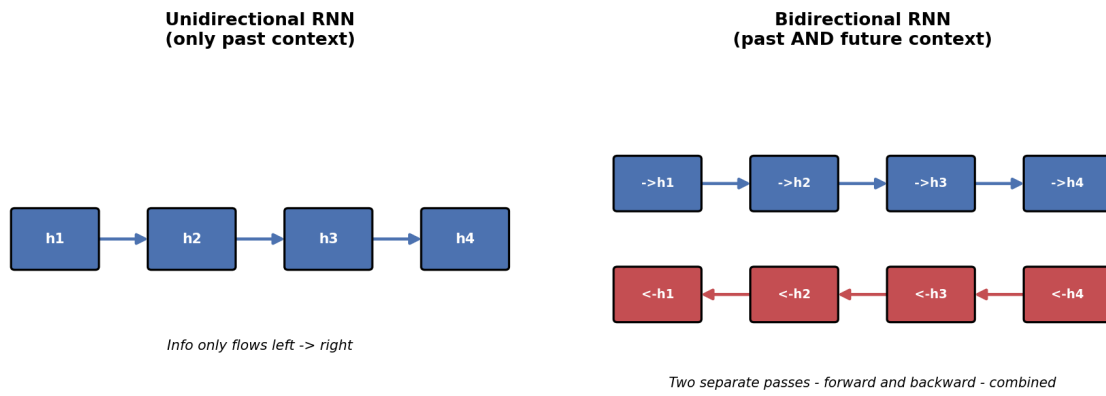


Figure 2: A unidirectional RNN only sees the past; a bidirectional RNN sees the past AND the future at every position.

3. Architecture

A Bidirectional RNN is really just **two separate RNN layers** running on the same input sequence:

- A **forward layer** that processes the sequence from $x(1)$ to $x(T)$, producing forward hidden states $h_{\text{fwd}}(1) \dots h_{\text{fwd}}(T)$.
- A **backward layer** that processes the same sequence in reverse, from $x(T)$ down to $x(1)$, producing backward hidden states $h_{\text{bwd}}(T) \dots h_{\text{bwd}}(1)$.
- At each time step t , the outputs of both layers are **merged** (most commonly by concatenation) to form the final representation used for prediction.

Bidirectional RNN - Full Architecture

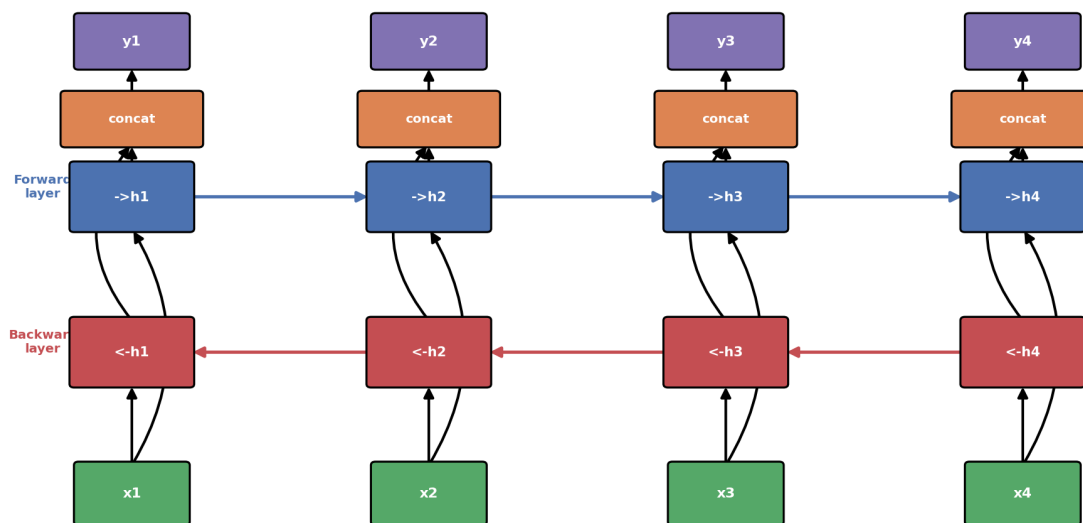


Figure 3: Full bidirectional architecture - forward layer (blue), backward layer (red), merged at every time step, then passed to the output layer.

$y(t) = g([h_{\text{fwd}}(t); h_{\text{bwd}}(t)])$ - where $[:]$ denotes concatenation and g is the output layer

Ways to Merge the Two Directions

Concatenation is most common, but a few merge strategies exist depending on the framework and use case:

Common Ways to Merge Forward & Backward Outputs

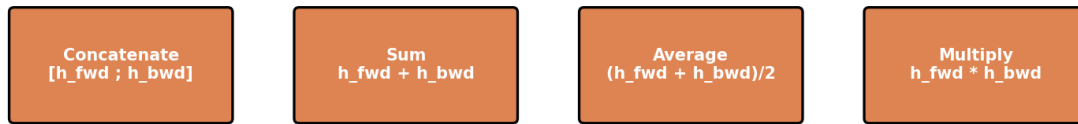


Figure 4: Common strategies for combining forward and backward hidden states.

Important: both layers are trained **jointly**, and crucially, the forward and backward layers do not share weights - they are two independent sets of parameters, effectively doubling the number of weights compared to a single-direction RNN of the same hidden size.

4. Advantages & Drawbacks

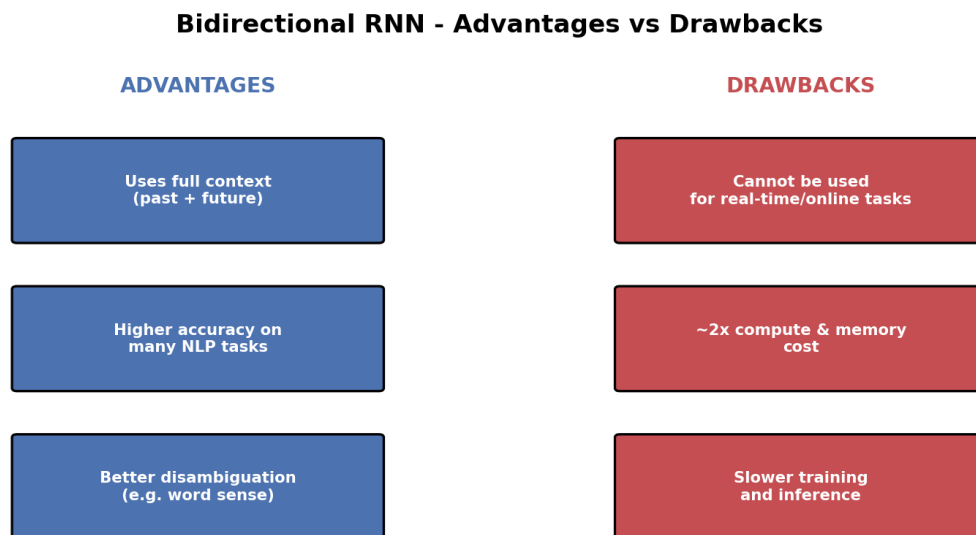


Figure 5: Key trade-offs to weigh before choosing a bidirectional architecture.

Advantages

- **Full context awareness** - every position has access to information from both before and after it.
- **Improved accuracy** on many sequence labeling and understanding tasks (NER, POS tagging, sentiment analysis, translation encoders).
- **Better disambiguation** of words/tokens whose meaning depends on later context.

Drawbacks

- **Not usable for real-time/streaming prediction** - the backward pass requires the entire sequence to already be available, so you can't use it to predict the next token as it arrives live (e.g. live speech transcription, real-time typing prediction).
- **Roughly double the computation and memory** compared to a unidirectional model with the same hidden size, since you're training two independent RNNs.
- **Slower training and inference** as a direct consequence of the doubled computation.

Rule of thumb: use bidirectional architectures whenever the **full sequence is available upfront** (e.g. classifying a complete sentence, tagging a finished document, encoding a sentence for translation). Avoid them for tasks that must generate or predict output token-by-token in real time, since future tokens simply don't exist yet.

5. BiLSTM & BiGRU

"Bidirectional" is a wrapper concept - it can be applied to **any** recurrent cell type, not just the vanilla RNN. In practice, the vanilla bidirectional RNN is rarely used on its own because of the vanishing gradient problem; instead, people use:

- **BiLSTM (Bidirectional LSTM)** - a forward LSTM and a backward LSTM, each with their full set of forget/input/output gates, running independently and merged at every step.
- **BiGRU (Bidirectional GRU)** - the same idea, but using GRU cells (reset/update gates) in each direction instead of LSTM cells.

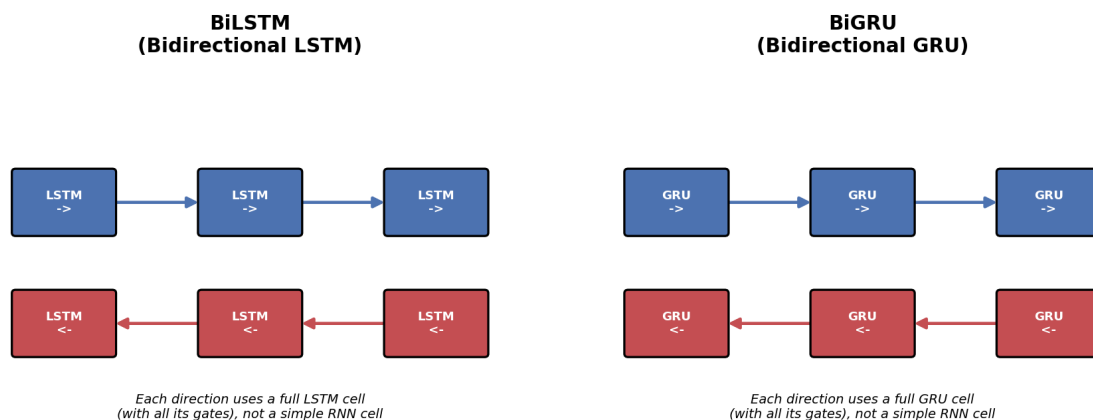


Figure 6: BiLSTM and BiGRU simply swap the plain RNN cell for a full LSTM or GRU cell in each direction.

Because LSTM and GRU cells already solve the vanishing gradient problem within each direction, BiLSTM and BiGRU combine the best of both worlds: strong long-range memory in each direction, plus full bidirectional context. This combination is extremely common in NLP encoders (e.g. as used historically in ELMo, and in many sequence-tagging and NER models) prior to the rise of Transformer-based architectures.

6. Summary

Model	Direction(s)	Cell used	Best for
RNN	Forward only	Simple tanh cell	Real-time / streaming tasks, short sequences
LSTM / GRU	Forward only	Gated cell	Real-time tasks needing long-range memory
Bidirectional RNN	Forward + Backward	Simple tanh cell	Rarely used alone (vanishing gradients)
BiLSTM	Forward + Backward	LSTM cell	Offline NLP tasks needing full context + long memory
BiGRU	Forward + Backward	GRU cell	Same as BiLSTM, faster to train, fewer parameters

Bottom line: bidirectional models trade extra compute for access to full-sequence context. When the whole input is available in advance and accuracy matters more than latency, BiLSTM or BiGRU is usually a strong, well-tested choice.